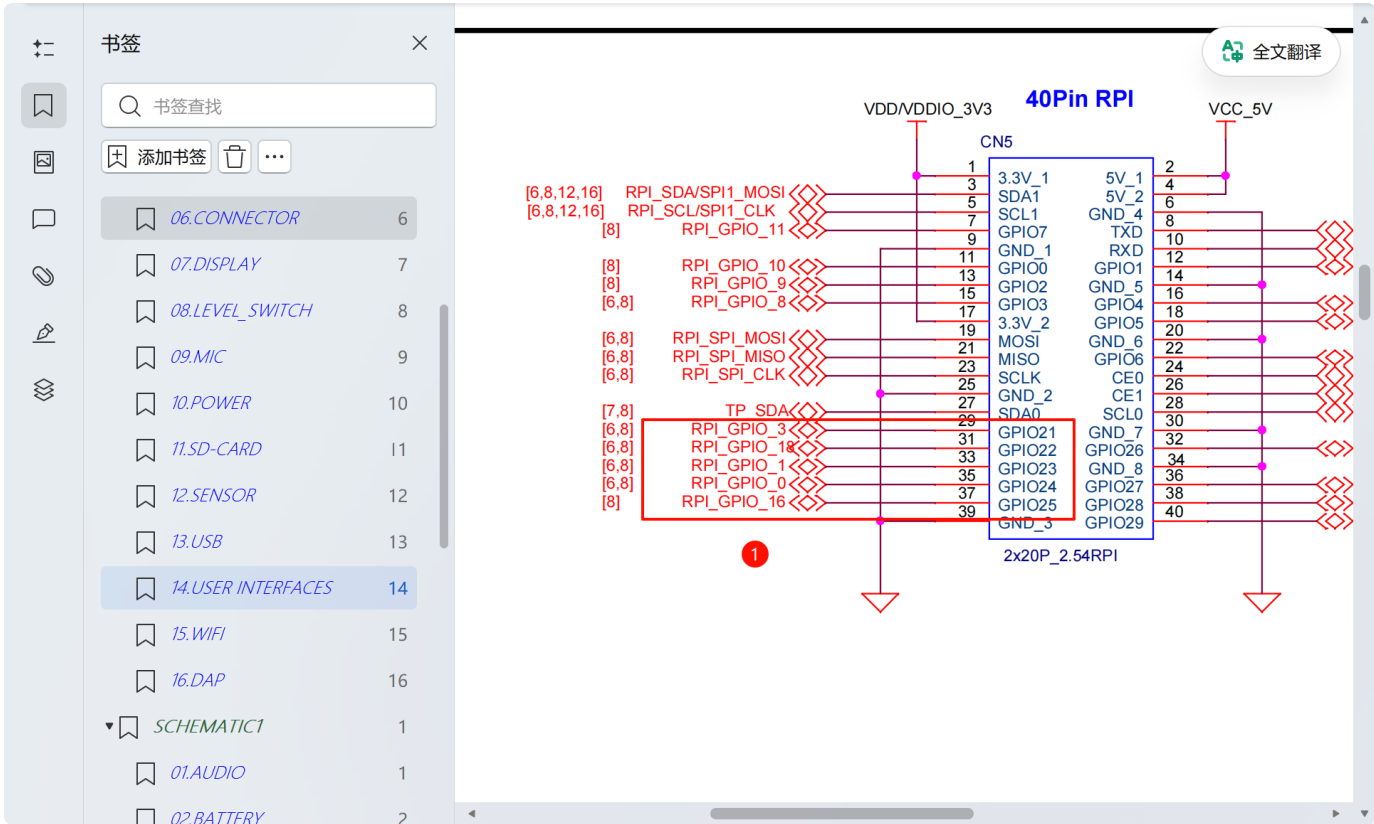


psoc-edge拓展外设配置手册

示范：如何添加额外的PWM外设

示范：如何添加额外的ADC外设

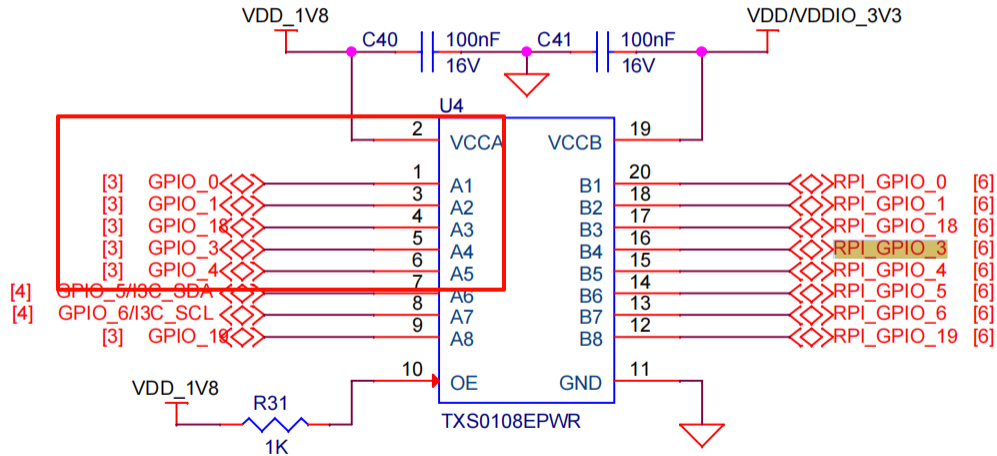
查找底板原理图，找到要配置的IO引脚



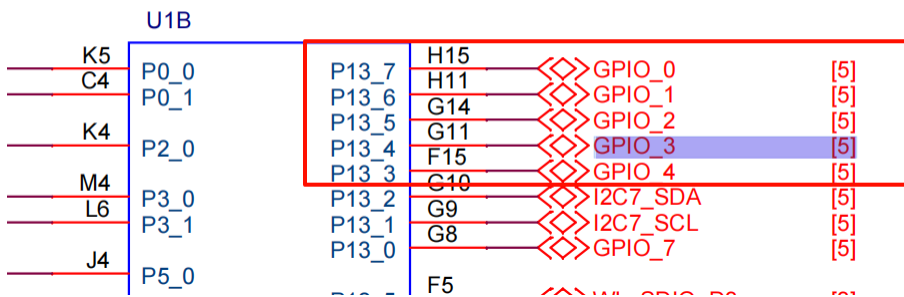
找到电源转换芯片对应的IO

Level switch

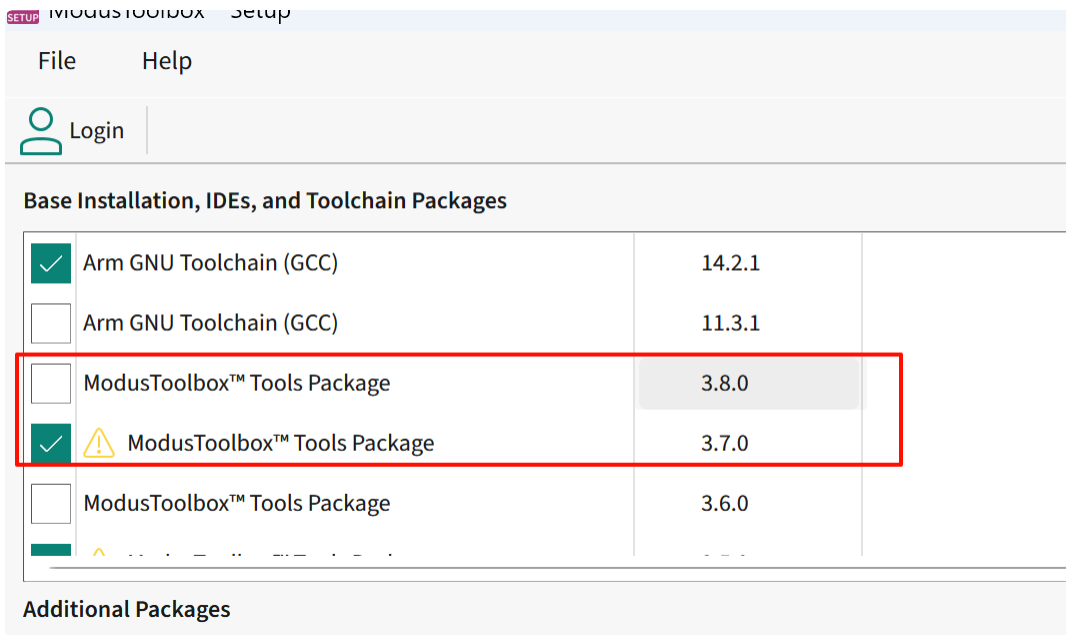
Level Translator for RPI Interface



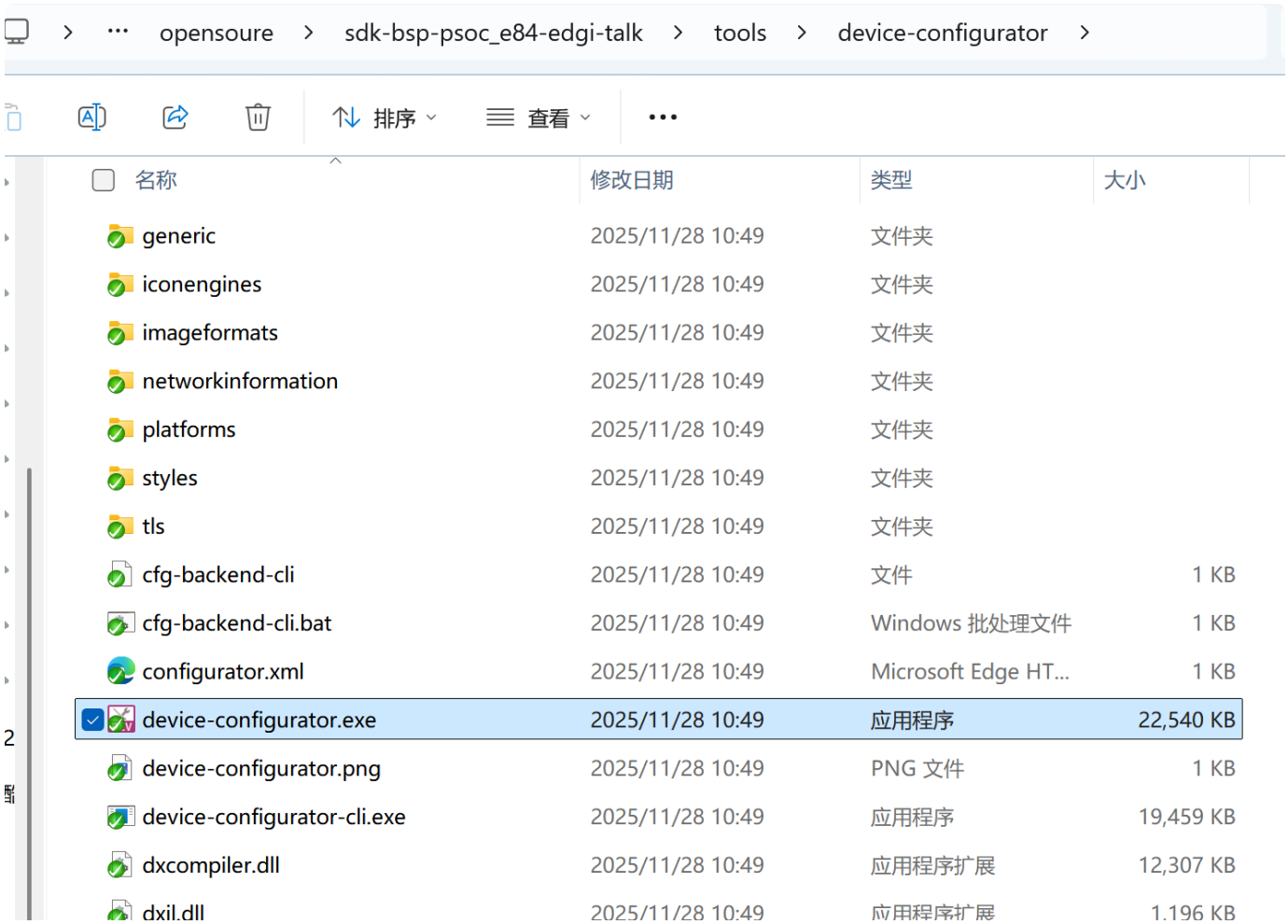
在核心板原理图找到对应的引脚定义



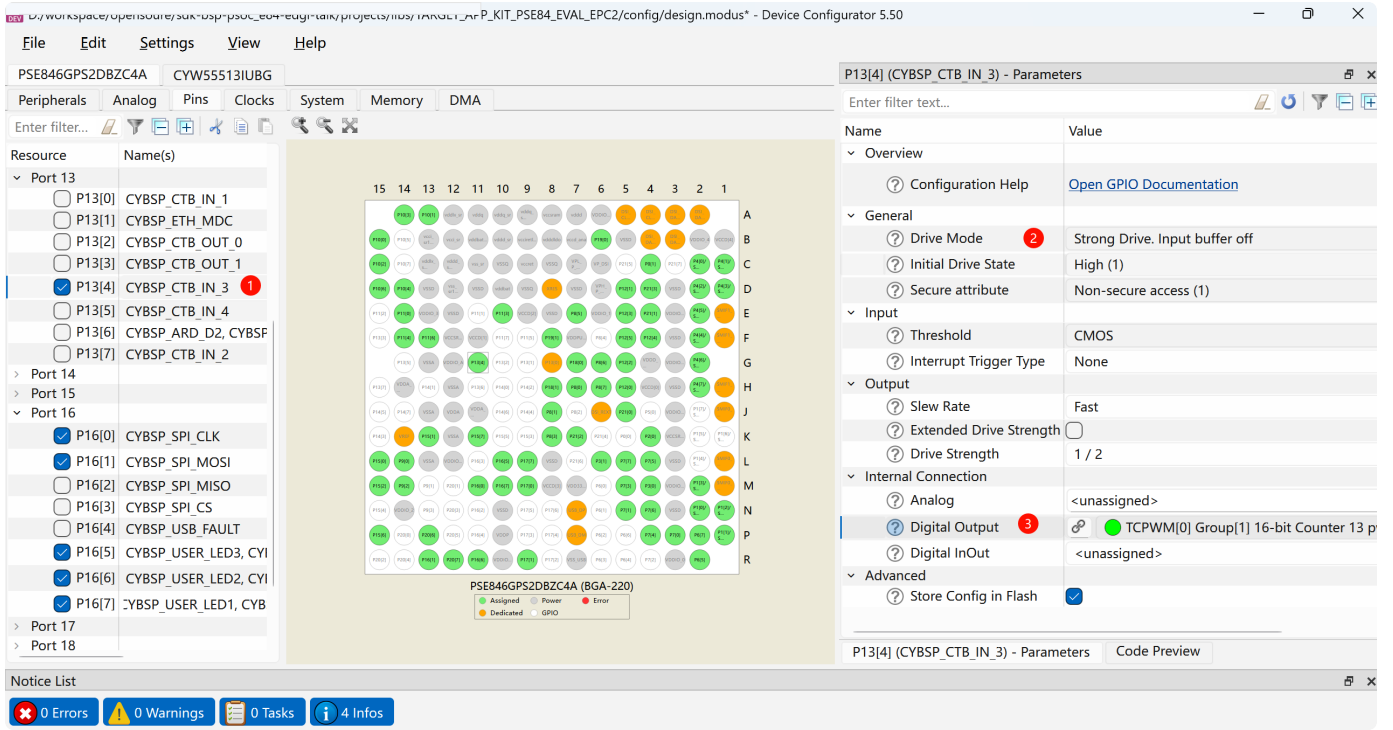
需要安装好英飞凌官方的 ModusToolbox™ Setup 软件，并安装>=3.5的Tools Package



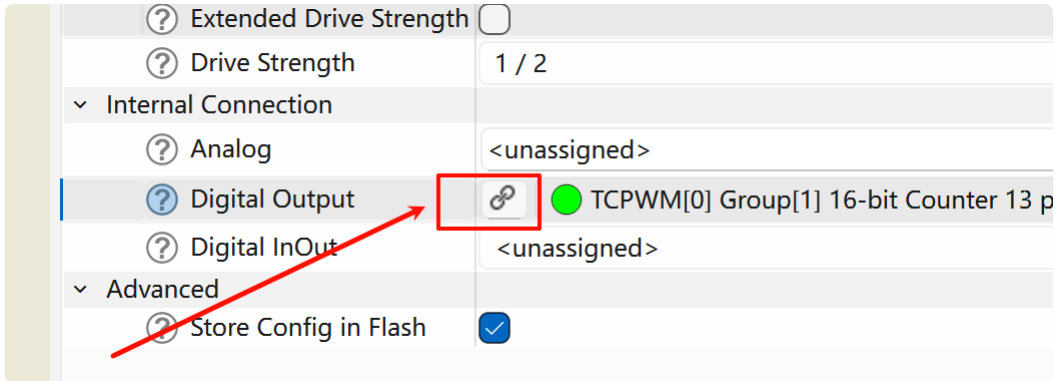
打开 device-configurator.exe 上位机，打开工程中的 `projects\libs\TARGET_APP_KIT_PSE84_EVAL_EPC2\config\design.modus` 配置文件进行配置



找到需要配置的IO，配置相关的工程，如下图配置 P13_4 引脚为PWM功能，需要配置IO的模式，输出的通道



点击链接到对应的PWM配置



使能PWM通道，进行 TCPWM[0] Group[1] 的相关的配置

📁

名称

📄

修改日期

📄

类型

📄

大小

📁

.hardware-config-server.generated-files

2026/5/13 18:24

GENERATED-FILES ...

1 KB

📄

cycfg.c

2026/5/13 18:24

C 源文件

2 KB

📄

cycfg.h

2026/5/13 18:24

C Header 源文件

3 KB

📄

cycfg.timestamp

2026/5/12 15:46

TIMESTAMP 文件

2 KB

📄

cycfg_clock_types.h

2026/5/13 18:24

C Header 源文件

1 KB

📄

cycfg_clocks.c

2026/5/13 18:24

C 源文件

25 KB

📄

cycfg_clocks.h

2026/5/13 18:24

C Header 源文件

5 KB

📄

cycfg_connectivity_bt.h

2026/5/13 18:24

C Header 源文件

3 KB

📄

cycfg_connectivity_wifi.h

2026/5/13 18:24

C Header 源文件

2 KB

📄

cycfg_memory.h

2026/5/13 18:24

C Header 源文件

3 KB

📄

cycfg_notices.h

2026/5/13 18:24

C Header 源文件

3 KB

📄

cycfg_peripheral_clocks.c

2026/5/13 18:24

C 源文件

8 KB

📄

cycfg_peripheral_clocks.h

2026/5/13 18:24

C Header 源文件

12 KB

📄

cycfg_peripherals.c

2026/5/13 18:24

C 源文件

72 KB

📄

cycfg_peripherals.h

2026/5/13 18:24

C Header 源文件

22 KB

📄

cycfg_pins.c

2026/5/13 18:24

C 源文件

45 KB

📄

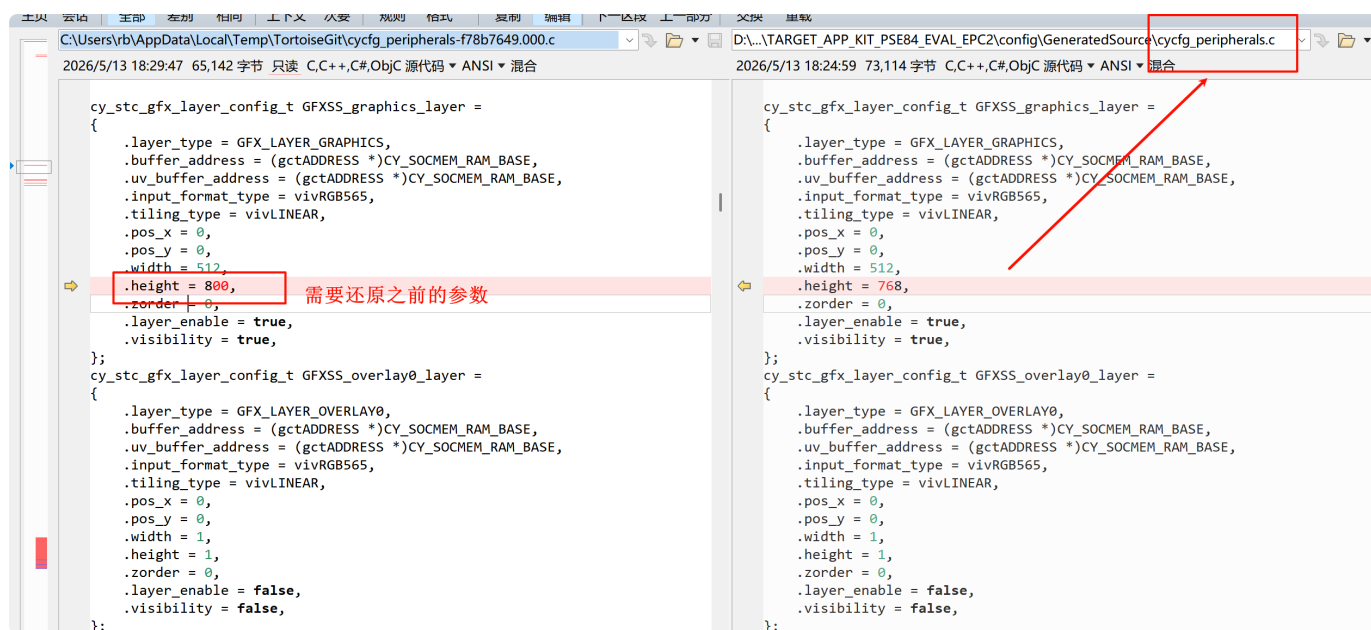
cycfg_pins.h

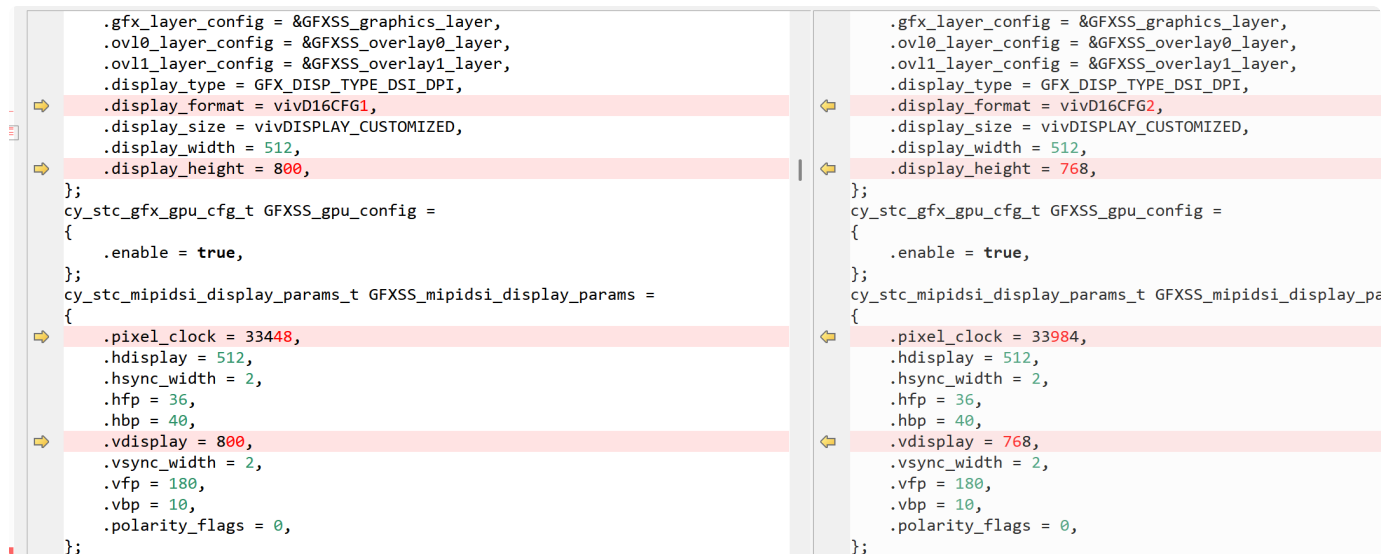
2026/5/13 18:24

C Header 源文件

54 KB

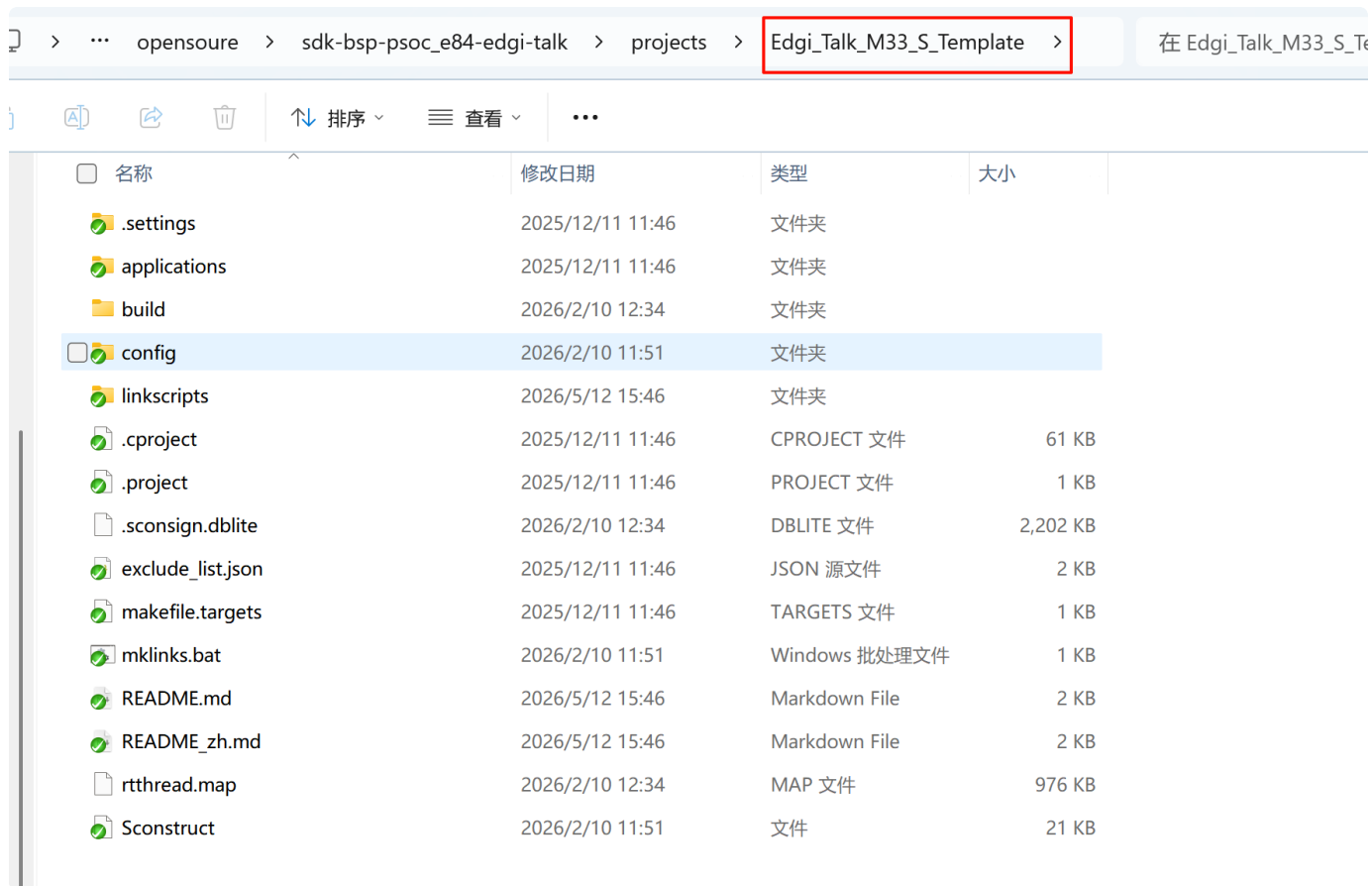
需要注意的是，我们还需要参考下面，还原下生成的屏幕参数配置（其余的不用管），否则后续屏幕使用将会异常：





示范：如何添加额外的PWM外设

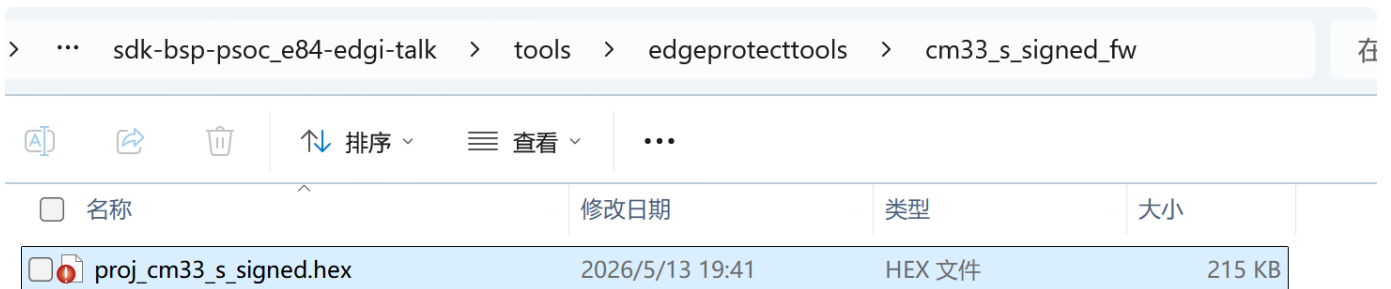
1. 重新编译 Edgi_Talk_M33_S_Template 工程，生成固件



```
CC build\libs_ext\TARGET_APP_KIT_PSE84_EVAL_EPC2\cybsp.o
CC build\libs_ext\TARGET_APP_KIT_PSE84_EVAL_EPC2\system_edge.o
LINK build\rtthread.elf
D:\workspace\work\Summer\env-windows\tools\bin\...\tools\gnu_gcc\arm_gcc\mingw\bin\arm-none-eabi-objcopy -O ihex build\rtthread.elf
build\rtthread.hex
D:\workspace\work\Summer\env-windows\tools\bin\...\tools\gnu_gcc\arm_gcc\mingw\bin\arm-none-eabi-size build\rtthread.elf
D:\workspace\work\Summer\env-windows\tools\bin\...\tools\gnu_gcc\arm_gcc\mingw\bin\arm-none-eabi-objcopy -O binary build\rtthread.elf
build\rtthread.bin
  text      data      bss      dec      hex filename
 71768    4164   131048   206980   32884 build\rtthread.elf
"D:\workspace\opensoure\sdk-bsp-psoc_e84-edgi-talk\tools\edgeprotecttools\bin\edgeprotecttools.exe" run-config -i "D:\workspace\opensoure\sdk-bsp-psoc_e84-edgi-talk\projects\Edgi_Talk_M33_S_Template\config\boot_with_extended_boot_scons.json"
: E : INFO : metadata_proj_cm33_s: command "sign" validation succeeded
: E : INFO : Image saved to 'D:\workspace\opensoure\sdk-bsp-psoc_e84-edgi-talk\projects\Edgi_Talk_M33_S_Template\build\rtthread.hex'
: E : INFO : metadata_proj_cm33_s: command "sign" succeeded
scons: done building targets.

(.venv) rb@RBB666 D:\workspace\opensoure\sdk-bsp-psoc_e84-edgi-talk\projects\Edgi_Talk_M33_S_Template
```

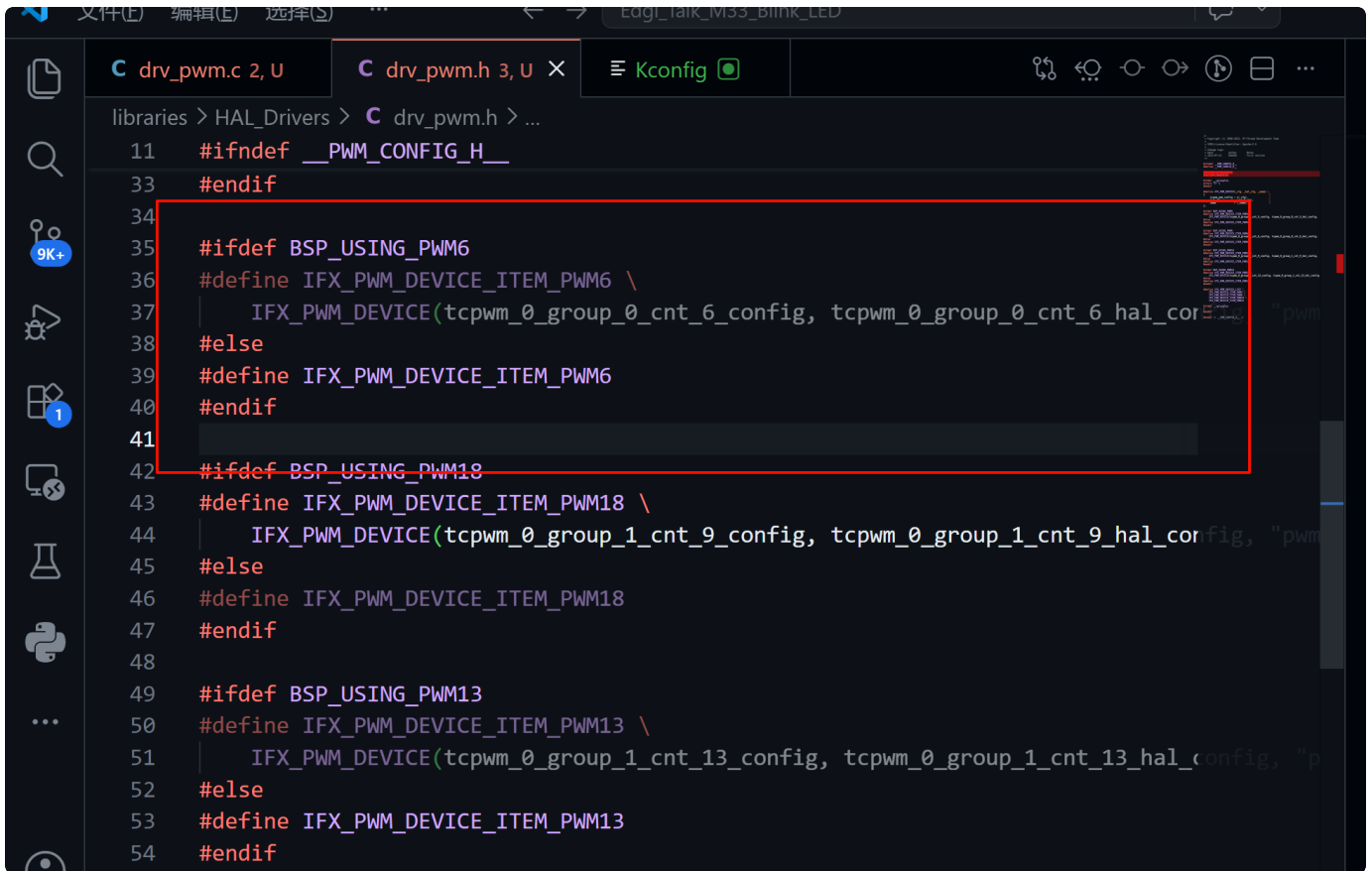
2. 拷贝 "projects\Edgi_Talk_M33_S_Template\build\rtthread.hex" 到 tools\edgeprotecttools\cm33_s_signed_fw 下, 重命名为 proj_cm33_s_signed.hex 进行替换



3. 打开M33对应的Kconfig文件, 添加新增的PWM通道, 这样在menuconfig后才能看到选项

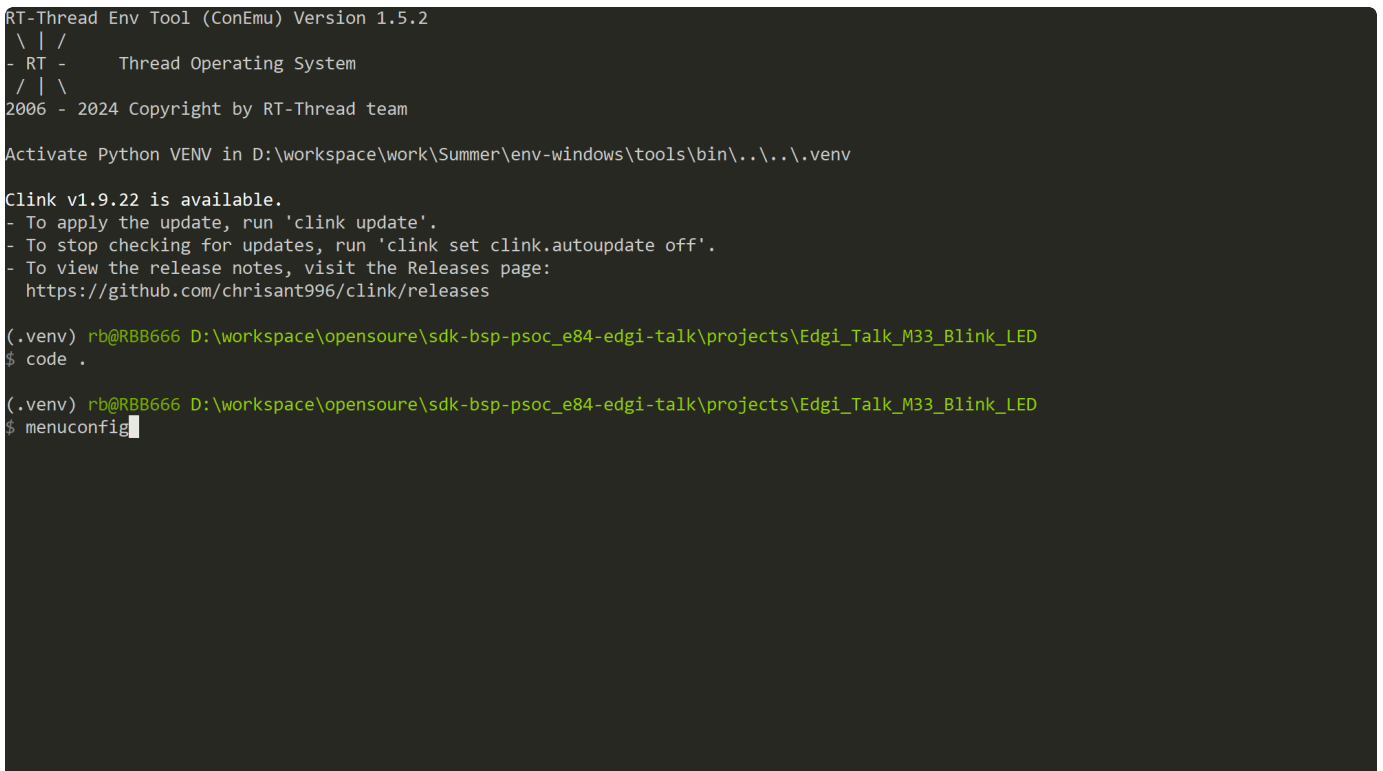
```
libraries > M33_Config > Kconfig
80  menu "On-chip Peripheral Drivers"
169  config BSP_USING_RTC
182  menuconfig BSP_USING_PWM
183      bool "Enable PWM"
184      select RT_USING_PWM
185      default n
186      if BSP_USING_PWM
187          config BSP_USING_PWM5
188              bool "Enable PWM5"
189              default n
190
191          config BSP_USING_PWM6
192              bool "Enable PWM6"
193              default n
194
195          config BSP_USING_PWM18
196              bool "Enable PWM18"
```


4. 在 drv_pwm.h 中添加新的PWM配置



```
libraries > HAL_Drivers > C drv_pwm.h > ...
11  #ifndef __PWM_CONFIG_H__
33  #endif
34
35  #ifdef BSP_USING_PWM6
36  #define IFX_PWM_DEVICE_ITEM_PWM6 \
37      IFX_PWM_DEVICE(tcpwm_0_group_0_cnt_6_config, tcpwm_0_group_0_cnt_6_hal_corfig, "pwm6")
38  #else
39  #define IFX_PWM_DEVICE_ITEM_PWM6
40  #endif
41
42  #ifdef BSP_USING_PWM18
43  #define IFX_PWM_DEVICE_ITEM_PWM18 \
44      IFX_PWM_DEVICE(tcpwm_0_group_1_cnt_9_config, tcpwm_0_group_1_cnt_9_hal_corfig, "pwm18")
45  #else
46  #define IFX_PWM_DEVICE_ITEM_PWM18
47  #endif
48
49  #ifdef BSP_USING_PWM13
50  #define IFX_PWM_DEVICE_ITEM_PWM13 \
51      IFX_PWM_DEVICE(tcpwm_0_group_1_cnt_13_config, tcpwm_0_group_1_cnt_13_hal_corfig, "pwm13")
52  #else
53  #define IFX_PWM_DEVICE_ITEM_PWM13
54  #endif
```

5. 打开env/RT-Thread Setting菜单



```
RT-Thread Env Tool (ConEmu) Version 1.5.2
\ | /
- RT - Thread Operating System
/ | \
2006 - 2024 Copyright by RT-Thread team

Activate Python VENV in D:\workspace\work\Summer\env-windows\tools\bin\...\venv

Clink v1.9.22 is available.
- To apply the update, run 'clink update'.
- To stop checking for updates, run 'clink set clink.autoupdate off'.
- To view the release notes, visit the Releases page:
  https://github.com/chrisant996/clink/releases

(.venv) rb@RBB666 D:\workspace\opensoure\sdk-bsp-psoc-e84-edgi-talk\projects\Edgi_Talk_M33_Blink_LED
$ code .

(.venv) rb@RBB666 D:\workspace\opensoure\sdk-bsp-psoc-e84-edgi-talk\projects\Edgi_Talk_M33_Blink_LED
$ menuconfig
```

```
(Top) → Hardware Drivers Config→ On-chip Peripheral Drivers→ Enable PWM
RT-Thread Configuration
```

```
[ ] Enable PWM5 (NEW)
[ ] Enable PWM6 (NEW)
[*] Enable PWM18 (NEW)
[ ] Enable PWM13 (NEW)
```

```
[Space/Enter] Toggle/enter  [ESC] Leave menu          [S] Save
[O] Load          [?] Symbol info        [/] Jump to symbol
[F] Toggle show-help mode  [C] Toggle show-name mode  [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)
```

6. 保存配置并退出

```
(Top)          g          s          M
RT-Thread Configuration
```

```
RT-Thread Kernel --->
RT-Thread Components --->
RT-Thread Utestcases --->
RT-Thread online packages --->
Hardware Drivers Config --->
[ ] Using USB with CherryUSB (NEW) ----
```

Quit

Save configuration?

(Y)es (N)o (C)ancel

```
[Space/Enter] Toggle/enter  [ESC] Leave menu          [S] Save
[O] Load          [?] Symbol info        [/] Jump to symbol
[F] Toggle show-help mode  [C] Toggle show-name mode  [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)
```

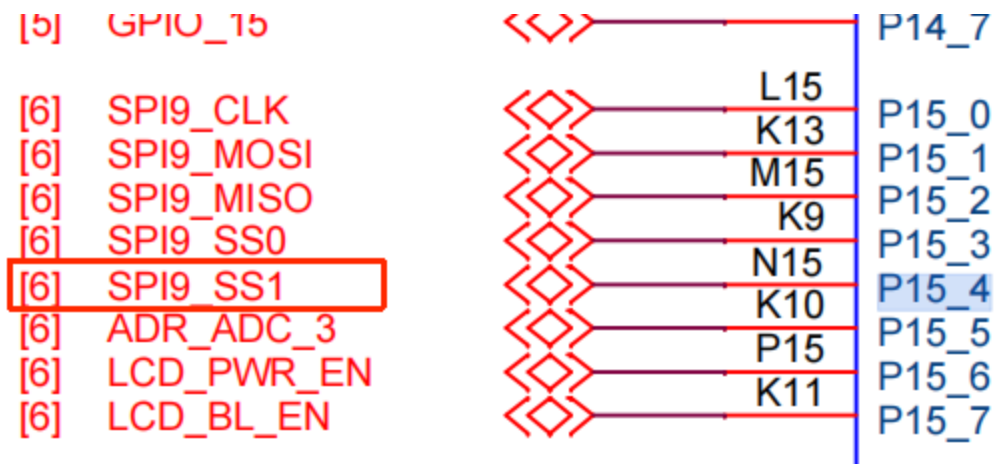
7. 使用scons进行编译

8. 使用 msh 命令进行测试：

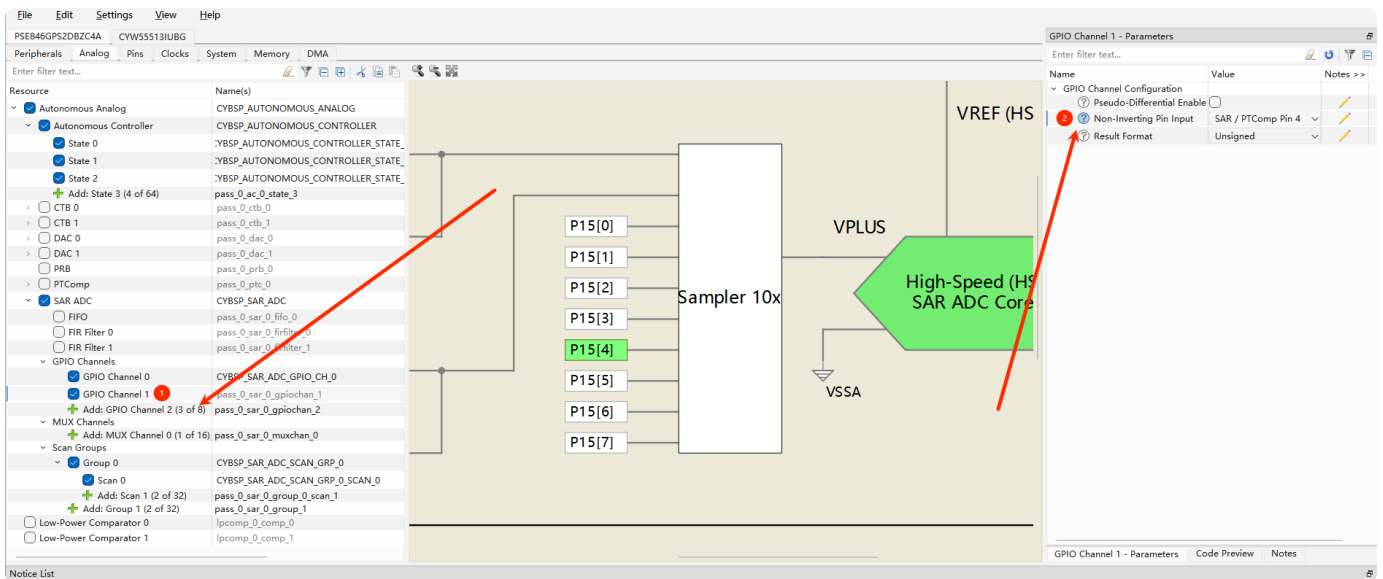
```
msh >list device
device          type          ref count
-----
pwm18           PWM Device      0
uart5           Character Device 0
uart2           Character Device 2
pin             Pin Device      0
msh >
msh >pwm probe pwm18 ①
probe pwm18 success
msh >
msh >pwm enable pwm18 ②
pwm18 channel 0 is enabled success
msh >
msh >pwm set 0 200000 150000 ③
pwm info set on pwm18 at channel 0
msh >
```

示范：如何添加额外的ADC外设

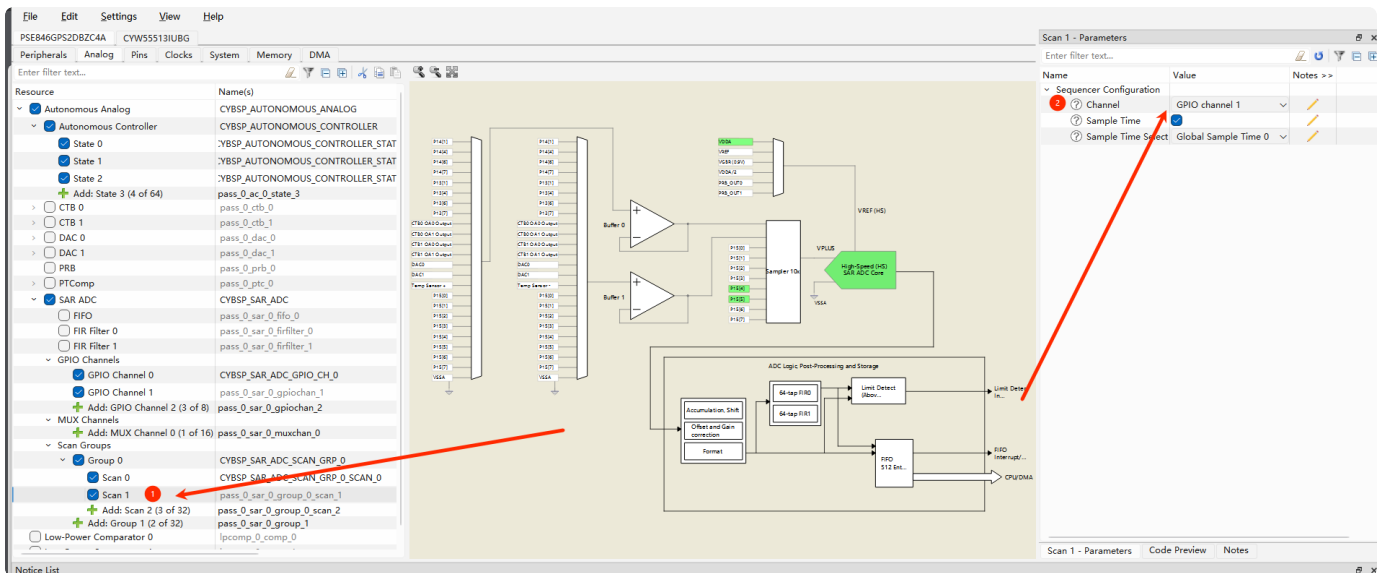
下面以配置 SPI9_SS1 为 ADC 功能为例：



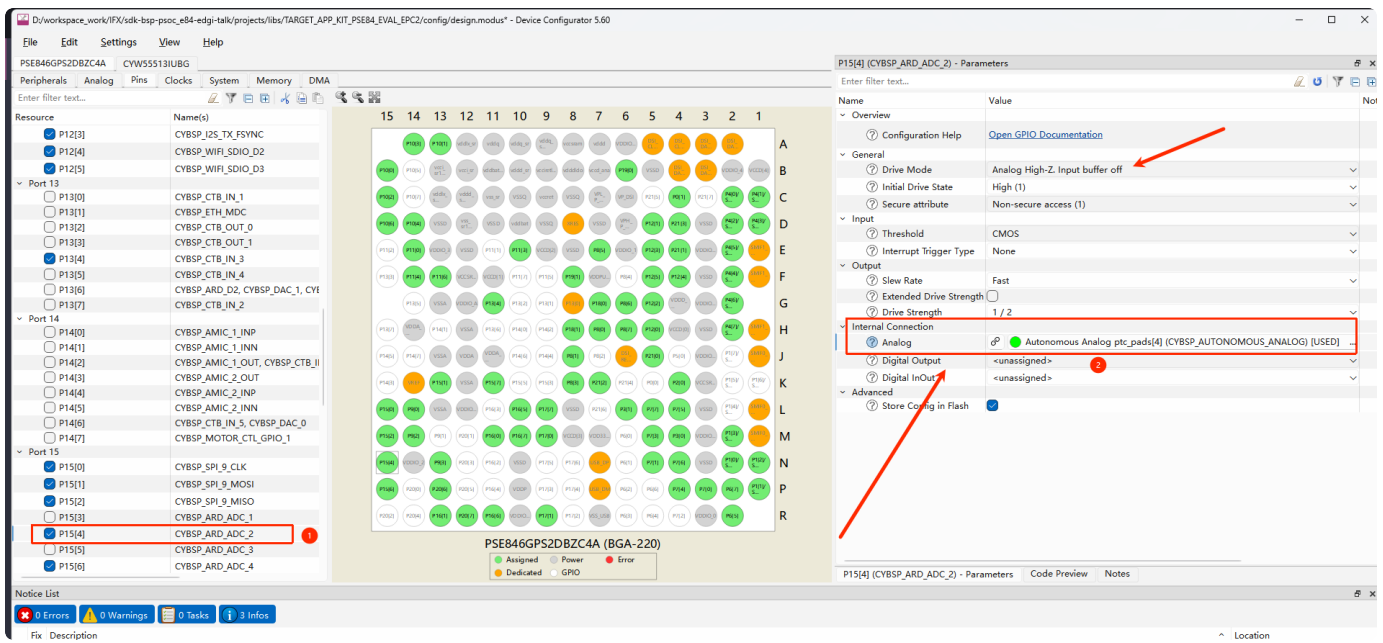
打开 device-configurator.exe 上位机，在 Analg 菜单栏中，新增一个GPIO Channel通道，然后配置IO为 Pin4，代表配置 P15[4] 引脚为ADC通道1功能。



配置 scan groups，新增一个扫描组，和上面的通道需要对应，因此这块配置为 channel 1

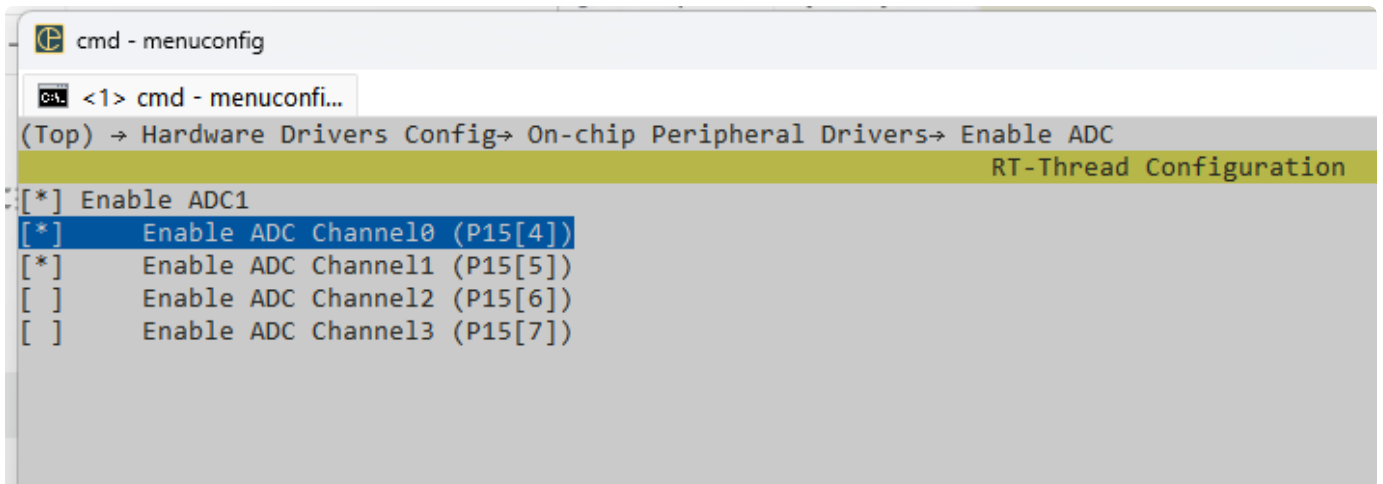


需要绑定IO引脚为具体的ADC功能



ctrl+s保存配置，生成底层代码，然后参考上面PWM的流程还原屏幕的配置代码。

在工程中使能对应的ADC通道



使用scons进行项目的构建，注意：需要重新编译并烧录M33的工程